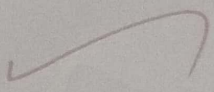


Program 1

Write a code in PL/SQL to develop a trigger that enforces referential integrity by preventing the deletion of a parent record if child records exist.

```
CREATE OR REPLACE TRIGGER trg-Prevent-parent-delete
BEFORE DELETE ON department
FOR EACH ROW

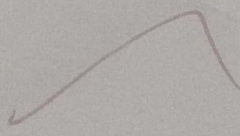
DECLARE
    v-count number;
BEGIN
    SELECT COUNT(*) INTO v-count FROM employee WHERE
        dept_id = OLD.dept_id;
    IF v-count > 0 THEN
        RAISE_APPLICATION_ERROR (-20001, 'cannot delete
parent record. child records exist in EMPLOYEE Table');
    END IF;
END;
/
```



Program 2

Write a code in PL/SQL to create a trigger that checks for duplicate values in a specific column and raises an exception if found.

```
CREATE OR REPLACE TRIGGER trg_check_duplicate_email
BEFORE INSERT OR UPDATE ON student
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT (*) INTO v_count FROM student WHERE email = NEW.email;
    IF v_count > 1 THEN
        RAISE_APPLICATION_ERROR (-20002, 'Duplicate email detected. Each email
must be unique.');
```



```
    END IF;
END;
```


Program 3

Write a code in PL/SQL to create a trigger that restricts the insertion of new rows if the total of a column's values exceeds a certain threshold.

```
CREATE OR REPLACE TRIGGER trg-limit-totalSalary
BEFORE INSERT ON employee
FOR EACH ROW
DECLARE
    v-total Number;
    v-threshold CONSTANT NUMBER := 10000;
BEGIN
    SELECT SUM(salary) INTO v-total FROM Employee;
    IF (v-total + :NEW salary) > v-threshold THEN
        RAISE_APPLICATION_ERROR (-20003, 'total salary exceeds all
        allowed threshold);
    END IF;
END;
/
```



Program 4

Write a code in PL/SQL to design a trigger that captures changes made to specific columns and logs them in an audit table.

```
CREATE TABLE employee-audit
```

```
emp-id NUMBER,
```

```
old-salary NUMBER,
```

```
new-salary NUMBER,
```

```
change-date DATE,
```

```
changed-by VARCHAR2 (30)
```

```
);
```

```
CREATE OR REPLACE TRIGGER trg-audit-salary-change
```

```
AFTER UPDATE OF salary ON employee
```

```
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO employee-audit (emp-id, old-salary, new-salary,  
change-date, changed-by)
```

```
VALUES (: OLD.emp-id, : OLD.salary, : NEW.salary, SYSDATE, USER
```

```
END;
```

```
/
```


Program 5

Write a code in PL/SQL to implement a trigger that records user activity (inserts, updates, deletes) in an audit log for a given set of tables.

```
CREATE TABLE activity-log (  
    table_name VARCHAR2(50),  
    operation_type VARCHAR2(20),  
    user_name VARCHAR(30),  
    activity_date DATE  
);
```

```
CREATE OR REPLACE TRIGGER trg-user-activity  
AFTER INSERT OR UPDATE OR DELETE ON employee  
BEGIN
```

```
    INSERT INTO activity-log (table_name, operation_type, user_  
        name, activity_date)
```

```
VALUES ('EMPLOYEE', 'ORA-SYSEVENT', USER, SYSDATE);
```

```
END;
```

```
/
```


Program 7

Write a code in PL/SQL to implement a trigger that automatically calculates and updates a running total column for a table whenever new rows are inserted.

```
CREATE TABLE sales
  sale_id NUMBER
  amount NUMBER
  running-total NUMBER);

CREATE OR REPLACE TRIGGER trg-update-running-total
AFTER INSERT ON sales
FOR EACH ROW
DECLARE
  v-total NUMBER;
BEGIN
  SELECT NVL (SUM (amount), 0) INTO v-total FROM sales;
  UPDATE sales SET running-total = v-total WHERE sale_id
    = : NEW.sale_id;
END;
/
```


Program 8

Write a code in PL/SQL to create a trigger that validates the availability of items before allowing an order to be placed, considering stock levels and pending orders.

```
CREATE OR REPLACE TRIGGER trg-check-stock-availability
BEFORE INSERT ON ORDERS
FOR EACH ROW
DECLARE
    v-stock NUMBER;
BEGIN
    SELECT quantity-in-stock INTO v-stock FROM inventory
    WHERE item-id = :NEW-item-id;

    IF v-stock < :NEW-order-quantity THEN
        RAISE_APPLICATION_ERROR(-20004, 'Insufficient stock available
        for the requested item);
    END IF;
END;
```



Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	5
Program/Execution (5)	5
Viva(5)	5
Total (15)	15
Faculty Signature	